Day 5

Agentic AI with LangGraph

Insurance Claims Workflow

Hands-on Enterprise Training Module for Insurance Automation Teams	
Duration	4 Hours / Half Day
Audience	UiPath Developers, RPA Developers, Automation Engineers, Solution Architects
Level	Beginner to Intermediate
Domain	Insurance Claims, Policy Validation, Document Review, Fraud Risk Routing
Final Outcome	Build a LangGraph-based Insurance Claim Workflow

Program Objective

Build practical understanding of Agentic AI and LangGraph by decomposing an insurance claim process into state, nodes, edges, conditional routing, and specialized agents. Participants implement a working claim workflow that can later connect with UiPath, APIs, policy systems, document systems, and human review queues.

4-Hour Session Plan

Time	Topic	Hands-on Focus	Outcome
30 min	Agentic AI Concepts	Insurance workflow thinking	Understand agentic automation boundaries
35 min	LangGraph Basics	State, nodes, edges	Create first graph structure
35 min	Claim Intake Agent	Extract claim data	Convert raw notes

Time	Topic	Hands-on Focus	Outcome
			into structured state
35 min	Document Validation Agent	Check missing documents	Create document checklist output
35 min	Policy Lookup Agent	Verify policy and premium	Attach policy decision factors
35 min	Fraud Risk Review Agent	Risk review and routing	Identify risk indicators safely
35 min	Conditional Routing	Approved, document pending, risk review	Build graph routing paths
20 min	Final Assignment	End-to-end workflow	Submit working workflow design

Agentic AI Concepts for Automation Engineers

Agentic AI means designing AI-enabled workflows where components can reason over context, decide next actions, call tools, and move work forward under defined boundaries. For UiPath developers, it is a smart process layer that chooses the next transaction path while respecting business rules, audit needs, and human approval points.

Concept	Practical Meaning
Traditional RPA	Follows predefined steps and rules.
Agentic AI	Uses context, state, and decisions to choose the next workflow path.
Insurance Example	A claim may go to document pending, policy exception, fraud review, or processor review.

LangGraph State, Nodes, Edges and Conditional Routing

LangGraph helps build stateful agent workflows. A graph contains a shared state object, processing nodes, edges between nodes, and conditional routing rules. This is useful because every insurance claim carries state: claim details, documents, policy status, risk score, and final route.

Concept	Practical Meaning
State	Shared workflow memory: claim_id, policy_number, documents, risk_level.
Node	A processing step such as claim intake or policy lookup.
Edge	Connection between steps.
Conditional Routing	Decision logic that sends claim to the correct next step.

Single-Agent versus Multi-Agent Design

A single-agent design uses one AI component to perform many tasks. A multi-agent design separates responsibilities across specialized agents. In enterprise insurance workflows, multi-agent design is easier to audit because each agent has a clear boundary.

Concept	Practical Meaning
Single Agent	Good for small demos and simple assistants.
Multi-Agent	Better for enterprise workflows with separate responsibilities.
Insurance Example	Claim Intake Agent, Document Agent, Policy Agent, Fraud Agent, Routing Agent.

Agent Responsibilities and Workflow Boundaries

Every agent should have a narrow responsibility. This reduces hallucination, improves quality, and makes the process easier to test. Agents should not make final claim payment decisions unless business policy explicitly allows it.

Concept	Practical Meaning
Claim Intake Agent	Extracts facts from raw claim note.
Document Validation Agent	Checks submitted and missing documents.
Policy Lookup Agent	Checks policy, premium, and coverage.
Fraud Risk Review Agent	Identifies risk indicators neutrally.

Concept	Practical Meaning
Routing Agent	Routes claim to correct queue based on state.

Insurance Claim Workflow Decomposition

A complete claim workflow can be decomposed into smaller steps. This allows developers to test each component independently before connecting them in a graph.

Concept	Practical Meaning
Input	Raw claim note or extracted document text.
Processing	Intake, documents, policy lookup, risk review.
Decision	Approve for review, pending documents, reject, escalate.
Output	Structured JSON for UiPath or claim system.

Lab Setup and Environment Configuration

Assumption: API key is provided to all participants and environment variables are already configured. Participants should verify the environment before running LangGraph examples.

Environment Verification Code

```
# Required packages
# pip install langgraph langchain langchain-openai openai python-dotenv pydantic

import os
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()

OPENROUTER_API_KEY = os.getenv("OPENROUTER_API_KEY")
OPENROUTER_BASE_URL = os.getenv("OPENROUTER_BASE_URL",
"https://openrouter.ai/api/v1")
OPENROUTER_MODEL = os.getenv("OPENROUTER_MODEL", "openai/gpt-4.1-mini")

print("API Key Loaded:", bool(OPENROUTER_API_KEY))
print("Base URL:", OPENROUTER_BASE_URL)
print("Model:", OPENROUTER_MODEL)

client = OpenAI(
```

```
    base_url=OPENROUTER_BASE_URL,
    api_key=OPENROUTER_API_KEY
)
```

Explanation: load_dotenv() loads variables from the .env file. os.getenv() reads each value safely. The client is configured with OpenRouter base URL and model. This avoids hard-coding keys inside notebooks or source files.

Reusable LLM Function

```
def call_llm(prompt, system_message="You are an insurance claims automation
assistant.", temperature=0.2):
    response = client.chat.completions.create(
        model=OPENROUTER_MODEL,
        messages=[
            {"role": "system", "content": system_message},
            {"role": "user", "content": prompt}
        ],
        temperature=temperature
    )
    return response.choices[0].message.content

# Quick test
# print(call_llm("Explain claim validation in one sentence."))
```

Insurance Sample Data for Labs
Sample Claim Note and Policy Database

```
raw_claim_note = """
Customer Rajesh Sharma submitted a health insurance claim on 12 June 2026.
He was admitted to City Care Hospital for dengue fever treatment.
Total bill amount is INR 85,000. Policy number is POL1001.
Submitted documents include hospital bill and discharge summary.
Doctor prescription is not attached. Claim is pending document verification.
"""

policy_database = {
    "POL1001": {
        "customer_name": "Rajesh Sharma",
        "policy_type": "Health",
        "coverage_amount": 500000,
        "policy_status": "Active",
        "premium_status": "Paid"
    },
    "POL2002": {
        "customer_name": "Meena Joshi",
        "policy_type": "Vehicle",
        "coverage_amount": 300000,
        "policy_status": "Active",
```

```
        "premium_status": "Pending"
    }
}
```

LangGraph Starter Structure

The first workflow uses a shared state dictionary. Each node receives the state, updates part of it, and returns the updated state. Conditional edges route the claim based on document, policy, and risk status.

Shared Claim State

```
from typing import TypedDict, List
from langgraph.graph import StateGraph, END

class ClaimState(TypedDict, total=False):
    raw_claim_note: str
    claim_id: str
    customer_name: str
    policy_number: str
    claim_type: str
    claim_amount: int
    submitted_documents: List[str]
    missing_documents: List[str]
    policy_status: str
    premium_status: str
    coverage_amount: int
    risk_level: str
    risk_indicators: List[str]
    route: str
    final_message: str
```

Explanation: ClaimState defines the fields that move through the workflow. It is similar to a shared transaction object in UiPath. Each agent updates only its responsibility area.

Hands-on Lab 1: Create a Claim Intake Agent

Objective: Extract structured claim details from raw claim notes.

1. Create a prompt to identify customer name, policy number, claim type, claim amount, and submitted documents.
2. Call the LLM using call_llm().
3. For beginner practice, first return readable text; later convert it to structured JSON.
4. Update the ClaimState with extracted fields.

Expected Output: State contains claim details ready for the next agent.

Hands-on Lab 2: Create a Document Validation Agent

Objective: Check submitted and missing documents for health or vehicle claim processing.

5. Define required document list for Health claim and Vehicle claim.
6. Compare submitted documents with required documents.
7. Add missing documents to state.
8. Create a short explanation for claim processor.

Expected Output: State contains missing_documents list.

Hands-on Lab 3: Create a Policy Lookup Agent

Objective: Verify policy status, premium status, and coverage amount.

9. Use policy_number from state.
10. Lookup policy in sample policy_database.
11. Update policy_status, premium_status, and coverage_amount.
12. Handle policy not found scenario.

Expected Output: State contains policy verification result.

Hands-on Lab 4: Create a Fraud Risk Review Agent

Objective: Identify fraud risk indicators using neutral language.

13. Create a prompt to review risk indicators.
14. Never accuse the customer directly.
15. Classify risk as Low, Medium, or High.
16. Add risk_level and risk_indicators to state.

Expected Output: State contains risk review result.

Hands-on Lab 5: Route Claims Based on Policy and Risk Status

Objective: Use conditional routing to send claim to correct queue.

17. If policy inactive or not found, route to policy exception.
18. If premium pending, route to payment follow-up.
19. If documents missing, route to document pending.
20. If high risk, route to fraud review.
21. Otherwise, route to claim processor review.

Expected Output: Final route is assigned for downstream UiPath or claim system.

Step-by-Step Code: Claim Intake Agent
Claim Intake Agent

```
def claim_intake_agent(state: ClaimState) -> ClaimState:
    prompt = f"""
    Extract the following fields from the insurance claim note:
    - customer_name
    - policy_number
    - claim_type
    - claim_amount
    - submitted_documents

    Return valid JSON only.

    Claim Note:
    {state['raw_claim_note']}
    """
    # response = call_llm(prompt)
    # Beginner-friendly demo values:
    state['claim_id'] = 'CLM1001'
    state['customer_name'] = 'Rajesh Sharma'
    state['policy_number'] = 'POL1001'
    state['claim_type'] = 'Health'
    state['claim_amount'] = 85000
    state['submitted_documents'] = ['hospital bill', 'discharge summary']
    return state
```

Trainer Tip: First run the hard-coded version so participants understand state update. Then replace hard-coded values with parsed LLM JSON output.

Step-by-Step Code: Document Validation Agent
Document Validation Agent

```
def document_validation_agent(state: ClaimState) -> ClaimState:
    required_docs = {
        'Health': ['hospital bill', 'discharge summary', 'doctor prescription'],
        'Vehicle': ['policy copy', 'driving license', 'vehicle registration',
'photos', 'repair estimate']
    }
    claim_type = state.get('claim_type', 'Health')
    submitted = [doc.lower() for doc in state.get('submitted_documents', [])]
    required = required_docs.get(claim_type, [])
    missing = [doc for doc in required if doc.lower() not in submitted]
    state['missing_documents'] = missing
    return state
```

Step-by-Step Code: Policy Lookup Agent
Policy Lookup Agent

```
def policy_lookup_agent(state: ClaimState) -> ClaimState:
    policy_number = state.get('policy_number')
    policy = policy_database.get(policy_number)
    if not policy:
        state['policy_status'] = 'Not Found'
        state['premium_status'] = 'Unknown'
        state['coverage_amount'] = 0
        return state
    state['policy_status'] = policy['policy_status']
    state['premium_status'] = policy['premium_status']
    state['coverage_amount'] = policy['coverage_amount']
    return state
```

Step-by-Step Code: Fraud Risk Review Agent
Fraud Risk Review Agent

```
def fraud_risk_review_agent(state: ClaimState) -> ClaimState:
    risk_indicators = []
    if state.get('claim_amount', 0) > state.get('coverage_amount', 0):
        risk_indicators.append('Claim amount exceeds coverage amount')
    if state.get('missing_documents'):
        risk_indicators.append('Required documents are missing')
    if len(risk_indicators) >= 2:
        risk_level = 'High'
    elif len(risk_indicators) == 1:
        risk_level = 'Medium'
    else:
        risk_level = 'Low'
    state['risk_indicators'] = risk_indicators
    state['risk_level'] = risk_level
    return state
```

Trainer Note: Risk indicators are not accusations. They are operational signals that may require review.

Step-by-Step Code: Conditional Routing Function
Conditional Routing Logic

```
def route_claim(state: ClaimState) -> str:
    if state.get('policy_status') == 'Not Found':
        return 'policy_exception'
    if state.get('policy_status') != 'Active':
        return 'policy_exception'
    if state.get('premium_status') != 'Paid':
```

```
        return 'payment_follow_up'
    if state.get('missing_documents'):
        return 'document_pending'
    if state.get('risk_level') == 'High':
        return 'fraud_review'
    return 'claim_processor_review'
```

Step-by-Step Code: Final Response Nodes
Final Queue Nodes

```
def policy_exception_node(state: ClaimState) -> ClaimState:
    state['route'] = 'Policy Exception Queue'
    state['final_message'] = 'Policy issue found. Route to policy operations team.'
    return state

def payment_follow_up_node(state: ClaimState) -> ClaimState:
    state['route'] = 'Payment Follow-up Queue'
    state['final_message'] = 'Premium is pending. Customer payment follow-up
required.'
    return state

def document_pending_node(state: ClaimState) -> ClaimState:
    state['route'] = 'Document Pending Queue'
    state['final_message'] = f"Missing documents: {state.get('missing_documents')}"
    return state

def fraud_review_node(state: ClaimState) -> ClaimState:
    state['route'] = 'Fraud Risk Review Queue'
    state['final_message'] = 'High risk indicators found. Route to fraud review
team.'
    return state

def claim_processor_review_node(state: ClaimState) -> ClaimState:
    state['route'] = 'Claim Processor Review Queue'
    state['final_message'] = 'Claim ready for processor review.'
    return state
```

Step-by-Step Code: Build the LangGraph Workflow
LangGraph Workflow

```
workflow = StateGraph(ClaimState)

workflow.add_node('claim_intake', claim_intake_agent)
workflow.add_node('document_validation', document_validation_agent)
workflow.add_node('policy_lookup', policy_lookup_agent)
workflow.add_node('fraud_risk_review', fraud_risk_review_agent)
```

```
workflow.add_node('policy_exception', policy_exception_node)
workflow.add_node('payment_follow_up', payment_follow_up_node)
workflow.add_node('document_pending', document_pending_node)
workflow.add_node('fraud_review', fraud_review_node)
workflow.add_node('claim_processor_review', claim_processor_review_node)

workflow.set_entry_point('claim_intake')
workflow.add_edge('claim_intake', 'document_validation')
workflow.add_edge('document_validation', 'policy_lookup')
workflow.add_edge('policy_lookup', 'fraud_risk_review')

workflow.add_conditional_edges(
    'fraud_risk_review',
    route_claim,
    {
        'policy_exception': 'policy_exception',
        'payment_follow_up': 'payment_follow_up',
        'document_pending': 'document_pending',
        'fraud_review': 'fraud_review',
        'claim_processor_review': 'claim_processor_review'
    }
)

for final_node in
['policy_exception','payment_follow_up','document_pending','fraud_review','claim_proce
ssor_review']:
    workflow.add_edge(final_node, END)

claim_graph = workflow.compile()
```

Step-by-Step Code: Run the Workflow

Run Claim Workflow

```
initial_state = {'raw_claim_note': raw_claim_note}
final_state = claim_graph.invoke(initial_state)

print('Final Route:', final_state.get('route'))
print('Final Message:', final_state.get('final_message'))
print('Risk Level:', final_state.get('risk_level'))
print('Missing Documents:', final_state.get('missing_documents'))
```

Mini Assignments for Practice

Assignment 1: Vehicle Claim Intake

Change the raw claim note to a vehicle accident claim. Extract policy number, claim amount, and submitted documents.

Assignment 2: Add Travel Claim Checklist

Extend document_validation_agent to support Travel insurance documents such as ticket, delay certificate, baggage report, and expense receipts.

Assignment 3: Improve Risk Rules

Add risk signals for high claim amount, recent policy start, missing police report, unclear photos, or repeated claims.

Assignment 4: Add Human Review Route

Create a new route called human_review for unclear, incomplete, or conflicting claim data.

Assignment 5: Return UiPath JSON

Create final output JSON with claim_id, route, risk_level, missing_documents, recommended_action, and audit_note.

Final Assignment / Outcome

Build a LangGraph-based Insurance Claim Workflow that accepts a raw claim note, extracts claim information, validates documents, checks policy and premium, reviews risk indicators, routes the claim, and produces a structured output for UiPath or a claim system.

Requirement	Expected Implementation	Completed
Claim Intake Agent	Extract claim facts from raw note	☐
Document Validation Agent	Identify submitted and missing documents	☐
Policy Lookup Agent	Check policy status, premium status, and coverage	☐
Fraud Risk Review Agent	Identify risk indicators and assign risk level	☐
Conditional Routing	Route to policy exception, payment follow-up, document pending, fraud review, or claim processor	☐

Final JSON Output	Generate UiPath-ready structured output	☐
Error Handling	Handle missing policy number, invalid claim amount, or empty document list	☐
Audit Note	Explain why the claim was routed to a queue	☐

Trainer Discussion Questions

- Which parts of the workflow should be deterministic rules and which parts can use LLM reasoning?
- Where should human approval be mandatory in claims processing?
- How can this LangGraph workflow be called from UiPath?
- What should be logged for audit and compliance?
- How can the same graph support health, vehicle, travel, and property claims?
- How should an enterprise team test each agent before production deployment?

Day 5 Summary

Participants learned how to decompose an insurance claim process into agentic workflow components using LangGraph. They created state, nodes, edges, conditional routing, and specialized agents for claim intake, document validation, policy lookup, fraud risk review, and final routing. The workflow can be extended into UiPath, FastAPI, LangChain, MCP tools, and enterprise claim systems.